

APObot a escala

Modelos locales y APIs para memoria, RAG, relaciones y razonamiento técnico

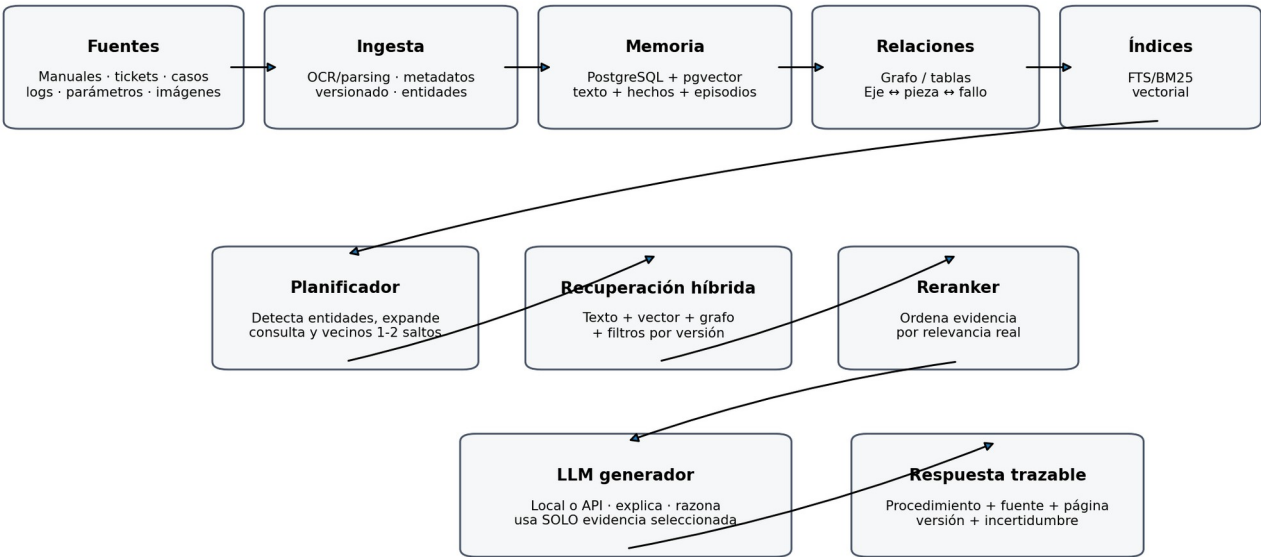
Estudio de coste-beneficio y arquitectura recomendada

Fecha de referencia: 17 de septiembre de 2026

Contexto: evolución de lruizap/apobot-knowledge-mvps hacia una base de conocimiento técnica de gran escala.

Arquitectura recomendada para APObot a escala

La “memoria” no debe depender de la ventana de contexto del modelo



Este documento separa dos problemas que suelen confundirse: (1) qué modelo genera la respuesta y (2) cómo el sistema encuentra y conserva el conocimiento. En una plataforma técnica grande, la calidad depende más de la segunda parte de lo que parece.

1. Resumen ejecutivo

Para APObot a gran escala no recomiendo elegir un único “modelo con mucha memoria”. La ventana de contexto de un LLM no sustituye una memoria persistente ni una estructura de relaciones. El sistema debe conservar hechos, episodios, documentos, versiones y conexiones entre componentes fuera del modelo, y recuperar solo la evidencia pertinente en cada consulta.

La arquitectura con mejor relación coste-beneficio para este proyecto es híbrida: recuperación y memoria locales, con un router de modelos que use un modelo económico para la mayoría de consultas y escale a un modelo más potente solo cuando la pregunta sea compleja, ambigua o de alto riesgo.

- Memoria y conocimiento: PostgreSQL + pgvector como base; full-text/BM25 + búsqueda vectorial + filtros por versión.
- Relaciones: modelar explícitamente eje → componente → síntoma → fallo → parámetro → procedimiento → evidencia. Neo4j puede aportar valor cuando esas relaciones crezcan; no debe añadirse solo por “tener un grafo”.
- Embeddings locales: Qwen3-Embedding-0.6B o 4B; para calidad máxima o multimodal, evaluar Gemini Embedding 2 o text-embedding-3-large.
- Reranking: Qwen3-Reranker o BGE reranker v2-m3 antes de enviar contexto al LLM.
- Generación por API: GPT-5.6 Luna / Mistral Small 4 para alto volumen; Claude Sonnet 5 o GPT-5.6 Terra como capa de mayor calidad; GPT-5.6 Sol / Claude Opus 5 solo para escalado selectivo.
- Generación local: Qwen3 14B para hardware moderado; Qwen3-30B-A3B o Qwen3 32B cuantizados cuando exista GPU de 24 GB o infraestructura equivalente.

Recomendación principal: empezar con un “APObot Hybrid” donde la memoria, recuperación, embeddings y reranking permanezcan bajo control propio y el generador sea intercambiable por configuración. Eso evita el lock-in y permite comparar OpenAI, Anthropic, Google, Mistral y modelos locales con la misma batería de casos.

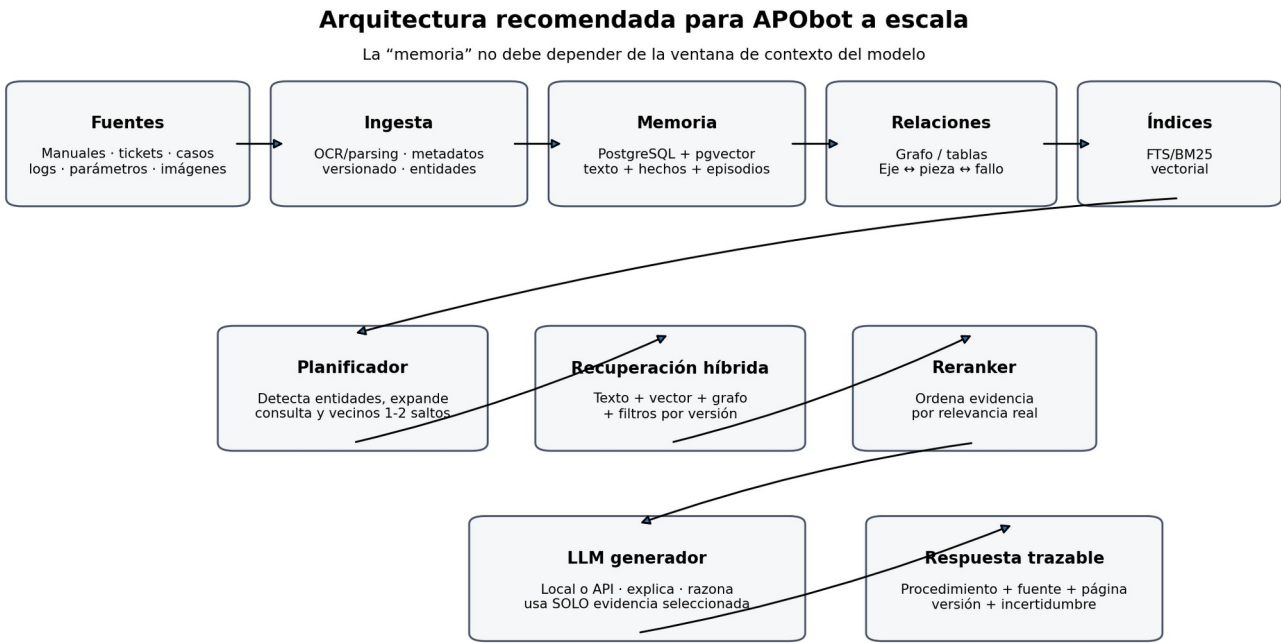
2. Qué significa realmente “escalar la memoria de IA”

En el MVP actual, el sistema responde sobre un único manual. A gran escala aparecerán cientos o miles de documentos, distintos modelos de robot, versiones de software, incidencias reales, fotografías, parámetros, piezas sustituidas, resultados de reparación y casos contradictorios. El reto deja de ser “buscar un párrafo” y pasa a ser “construir el contexto correcto”.

Necesidad	Qué debe hacer el sistema	No basta con
Entender un eje	Reconocer S-Achse como entidad y recuperar parámetros, límites, barrera, motores, correas, incidencias y procedimientos relacionados.	Una búsqueda literal por “eje S”.
Relacionar conceptos	Recorrer relaciones explícitas y semánticas: eje → pieza → síntoma → avería → solución → versión.	Un vector parecido semánticamente.
Memoria duradera	Recordar casos técnicos validados, resultados, versiones y cambios con procedencia.	El historial de chat.
Explicar bien	Sintetizar la evidencia en pasos claros, con condiciones y advertencias.	Un modelo grande sin fuentes.

Necesidad	Qué debe hacer el sistema	No basta con
No inventar	Rechazar o escalar cuando la evidencia sea insuficiente o conflictiva.	Un prompt que diga “no alucines”.
Escalar	Mantener latencia y coste razonables con millones de chunks.	Enviar todos los documentos al contexto del LLM.

3. Arquitectura recomendada para una base técnica grande



3.1 Capas de memoria

Capa	Contenido	Ejemplo APObot	Persistencia recomendada
Memoria documental	Manuales, PDFs, instrucciones, tablas, capturas y revisiones.	Manual de parametrización S/B/P v3.	Object storage + PostgreSQL/pgvector.
Memoria semántica	Conceptos y hechos extraídos de fuentes.	“KG+ de S = 340” con versión y fuente.	PostgreSQL estructurado.
Memoria episódica	Casos reales de mantenimiento y su resultado.	Síntoma → diagnóstico → intervención → resultado.	Tabla de casos/eventos.
Memoria relacional	Dependencias entre entidades.	S-Achse → LichtSchranke → parámetro → procedimiento.	Relaciones SQL o Neo4j.
Memoria conversacional	Objetivo actual, máquina, farmacia, versión y pasos ya realizados.	“Estamos reparando eje S del robot X”.	Redis/PostgreSQL con TTL + resumen.

Regla clave: cada recuerdo técnico debe tener fuente, fecha/versión, ámbito, confianza y estado de validación. Una “memoria” sin procedencia puede convertir una corrección antigua en una falsa verdad permanente.

3.2 Cómo resolver “me falla el eje S”

1. Detectar entidades e intención: eje S, tipo de fallo, modelo/versión del robot y contexto operativo.

- 2. Expandir la consulta con el vecindario relacional del eje S: límites, Nullpunkt, barrera de luz, motor, correa, lanza, parámetros, averías históricas y procedimientos vinculados.
- 3. Recuperar candidatos por tres vías: full-text/BM25, embeddings y grafo/relaciones.
- 4. Aplicar filtros duros: versión, modelo, fecha, estado “validado”, tipo de documento y seguridad.
- 5. Rerankear los candidatos con un modelo de relevancia que vea pregunta + fragmento.
- 6. Construir un contexto pequeño y de alta calidad, no un volcado de toda la base.
- 7. Pedir al LLM que explique la solución con fuentes, condiciones, discrepancias y nivel de confianza.

4. Modelos locales gratuitos / open-weight

Los modelos locales eliminan el coste por token y mejoran control y privacidad, pero el coste se desplaza a hardware, electricidad, despliegue, cuantización, monitorización y latencia. Para este proyecto, interesa más un modelo local sólido de 14B–32B bien alimentado por RAG que un modelo gigantesco mal recuperado.

Modelo	Tamaño / contexto	Fortaleza para APObot	Hardware práctico orientativo	Valoración coste-beneficio
Qwen3 8B	8B; 128K oficial [S6]	Buen baseline local, español razonable, herramientas y razonamiento.	Q4 ~5,2 GB de pesos; viable en equipos modestos.	Muy bueno para desarrollo, clasificación y routing.
Qwen3 14B	14B; 128K oficial [S6]	Mejor comprensión y síntesis que 8B manteniendo despliegue realista.	Q4 ~9,3 GB en Ollama; ideal con ~12–16 GB VRAM o RAM suficiente.	Mi opción local equilibrada para empezar.
Qwen3-30B-A3B	30B total / 3B activos; 128K oficial; variantes Ollama 256K [S6]	MoE eficiente; buena opción para respuestas técnicas más complejas.	Q4 ~19 GB; GPU 24 GB es un objetivo cómodo.	Excelente si ya existe hardware de 24 GB.
Qwen3 32B	32B; 128K oficial [S6]	Denso y más pesado; útil para comparar con 30B-A3B.	Q4 ~20 GB; 24 GB VRAM o más.	Bueno, pero el 30B-A3B puede ser más atractivo operativamente.
Gemma 3 12B	12B; 128K; multimodal [S8]	Documentos e imágenes; despliegue local razonable.	GPU media o CPU/RAM generosa según cuantización.	Alternativa sólida para corpus con capturas.
Gemma 3 27B	27B; 128K; multimodal [S8]	Mayor capacidad y visión local.	Clase 24 GB VRAM según cuantización/contexto.	Interesante si la imagen técnica será importante.
Mistral Small 4	119B total / 6,5B activos; 256K; Apache 2.0 [S5]	Modelo abierto moderno con tools y multimodalidad.	Los pesos totales lo hacen menos cómodo en una sola GPU de consumo.	Más interesante vía API o servidor dedicado.

Nota de hardware: el tamaño de un archivo cuantizado no equivale a la VRAM total necesaria. KV cache, contexto, runtime y concurrencia añaden memoria. Los valores anteriores sirven para dimensionamiento inicial, no como garantía de rendimiento.

4.1 Modelos de embeddings y reranking locales

Componente	Opción	Datos relevantes	Uso recomendado
Embedding	Qwen3-Embedding-0.6B	32K, 1024 dimensiones, multilingüe e instruction-aware [S7].	CPU/GPU ligera; indexación barata y continua.
Embedding	Qwen3-Embedding-4B	32K, 2560 dimensiones, multilingüe [S7].	Mejor candidato local cuando la calidad de recuperación prima.

Componente	Opción	Datos relevantes	Uso recomendado
Embedding	Qwen3-Embedding-8B	32K, 4096 dimensiones [S7].	Benchmark de máxima calidad local si el coste de indexación es aceptable.
Reranker	Qwen3-Reranker 0.6B/4B/8B	32K, cross-encoder [S7].	Reordenar 20–100 candidatos antes de generar.
Reranker	BGE-reranker-v2-m3	Multilingüe; modelo ~2,27 GB; hasta ~8K posiciones [S9].	Alternativa madura y sencilla para reranking local.

5. Modelos de API de pago

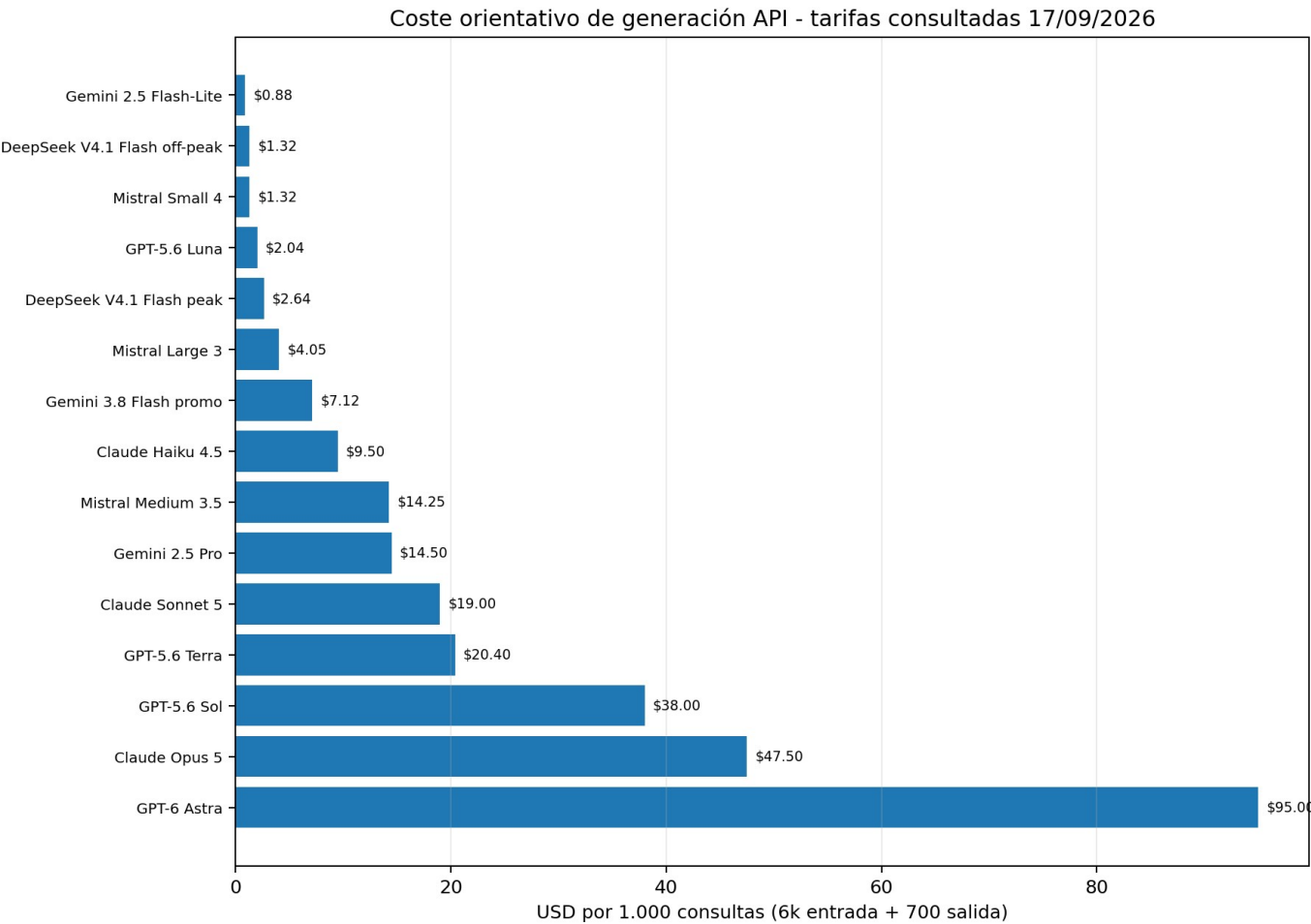
Para una aplicación real no se usa la suscripción de ChatGPT o Claude como backend: se usa la API del proveedor. Conviene que APObot dependa de una interfaz propia (por ejemplo, IGenerationModel) para poder cambiar de proveedor sin reescribir recuperación ni memoria.

Proveedor / modelo	Precio entrada / salida por 1M tokens	Contexto	Papel recomendado en APObot
OpenAI GPT-5.6 Luna	\$0,20 / \$1,20 [S1]	1,05M	Nivel 1 de gran volumen; candidato fuerte para RAG normal y routing.
OpenAI GPT-5.6 Terra	\$2 / \$12 [S1]	1,05M	Nivel 2 para preguntas ambiguas, multietapa y síntesis más exigente.
OpenAI GPT-5.6 Sol	\$4 / \$20 [S1]	1,05M	Nivel 3 selectivo. Precio promocional anunciado al menos hasta 21/11/2026.
OpenAI GPT-6 Astra	\$10 / \$50 [S12]	1,05M	Máxima capacidad; solo casos excepcionales, benchmark y tareas de alto valor.
Anthropic Claude Haiku 4.5	\$1 / \$5 [S3]	200K	Tier rápido; útil si Claude gana el benchmark de latencia/calidad.
Anthropic Claude Sonnet 5	\$2 / \$10 [S3]	1M	Nivel 2; candidato principal para razonamiento técnico/documental.
Anthropic Claude Opus 5	\$5 / \$25 [S3]	1M	Nivel 3; casos excepcionales y verificación de máxima exigencia.
Google Gemini 2.5 Flash-Lite	\$0,10 / \$0,40 [S4]	1M	Uno de los costes más bajos; excelente candidato de Nivel 1 si supera el benchmark APObot.
Google Gemini 3.8 Flash	\$0,75 / \$3,75 hasta 31/12/2026 [S4]	1M	Flash de nueva generación; precio promocional temporal.
Google Gemini 2.5 Pro	\$1,25 / \$10 (<=200K) [S4]	1M	Candidato equilibrado para Nivel 2; sube de precio con prompts >200K.
Mistral Small 4	\$0,15 / \$0,60 [S5]	256K	API muy barata y open-weight; baseline excelente de coste.
Mistral Large 3	\$0,50 / \$1,50 [S5]	Consultar ficha vigente	Alternativa económica de propósito general.
Mistral Medium 3.5	\$1,50 / \$7,50 [S5]	Consultar ficha vigente	Candidato de Nivel 2 para agentes y tool-calling.

Proveedor / modelo	Precio entrada / salida por 1M tokens	Contexto	Papel recomendado en APObot
DeepSeek V4.1 Flash	\$0,15 / \$0,60 off-peak; \$0,30 / \$1,20 peak [S13]	1M	Benchmark económico; revisar privacidad, contrato y residencia de datos antes de producción industrial.

6. Comparativa de coste real por consulta

Para comparar proveedores de forma útil, uso una consulta RAG de referencia con 6.000 tokens de entrada (instrucciones + memoria + evidencia recuperada) y 700 tokens de salida. No incluye herramientas externas, búsqueda web, almacenamiento ni caché. Los precios son una fotografía de las tarifas oficiales consultadas el 17/09/2026; promociones, batch, caché y residencia regional pueden alterar el coste.



Modelo	1.000 consultas	50.000 consultas	500.000 consultas
Gemini 2.5 Flash-Lite	\$0.88	\$44	\$440
DeepSeek V4.1 Flash off-peak	\$1.32	\$66	\$660
Mistral Small 4	\$1.32	\$66	\$660
GPT-5.6 Luna	\$2.04	\$102	\$1 020
DeepSeek V4.1 Flash peak	\$2.64	\$132	\$1 320
Mistral Large 3	\$4.05	\$202.5	\$2 025

Modelo	1.000 consultas	50.000 consultas	500.000 consultas
Gemini 3.8 Flash (promo 2026)	\$7.13	\$356.3	\$3 563
Claude Haiku 4.5	\$9.50	\$475	\$4 750
Mistral Medium 3.5	\$14.3	\$712.5	\$7 125
Gemini 2.5 Pro <=200K	\$14.5	\$725	\$7 250
Claude Sonnet 5	\$19	\$950	\$9 500
GPT-5.6 Terra	\$20.4	\$1 020	\$10 200
GPT-5.6 Sol	\$38	\$1 900	\$19 000
Claude Opus 5	\$47.5	\$2 375	\$23 750
GPT-6 Astra	\$95	\$4 750	\$47 500

Interpretación: un router que resuelva el 80–95% de las preguntas con un modelo económico y solo escale el resto puede mantener una calidad alta con un coste muy inferior al de usar siempre un modelo premium.

7. Mejor opción coste-beneficio por escenario

Escenario	Opción recomendada	Por qué	Alternativas
Desarrollo / pruebas 100% local	Qwen3 14B + Qwen3-Embedding-0.6B/4B + reranker local	Sin coste por token; suficiente para validar recuperación, memoria y UX.	Gemma 3 12B si la visión local pesa mucho.
Local serio / on-premise	Qwen3-30B-A3B + Qwen3-Embedding-4B + Qwen3-Reranker-4B	Mejor capacidad sin saltar a modelos enormes; encaja en hardware cuantizado de gama alta.	Qwen3 32B o Gemma 3 27B.
API de mínimo coste	Benchmark: Gemini 2.5 Flash-Lite / Mistral Small 4 / GPT-5.6 Luna / DeepSeek V4.1 Flash	El precio por token ya es muy bajo; debe ganar el que logre menor coste por respuesta correcta y trazable.	No decidir solo por precio unitario.
API equilibrada de alta calidad	A/B: Claude Sonnet 5 / GPT-5.6 Terra / Gemini 2.5 Pro	Rango intermedio para preguntas complejas; elegir con benchmark APObot.	Mistral Medium 3.5.
Máxima capacidad selectiva	GPT-5.6 Sol / Claude Opus 5 / GPT-6 Astra	Reservarlos para escalados donde complejidad o riesgo justifiquen el coste.	No usarlos como default.
Producción recomendada	Retrieval local + router: barato -> intermedio -> premium	Minimiza coste, mantiene la memoria bajo control y evita dependencia de un único proveedor.	Todo local cuando privacidad/offline pesen más que calidad punta.

Si hoy diseñara el siguiente MVP, mantendría conocimiento y memoria en PostgreSQL/pgvector, usaría embeddings y reranking locales, y mediría cuatro candidatos baratos para Nivel 1 (Gemini 2.5 Flash-Lite, Mistral Small 4, GPT-5.6 Luna y DeepSeek V4.1 Flash). Para Nivel 2 probaría Claude Sonnet 5, GPT-5.6 Terra y Gemini 2.5 Pro. Los modelos premium quedarían como fallback de baja frecuencia.

8. El modelo no debe ser quien “recuerde”

Un contexto de 1M de tokens ayuda a analizar grandes cantidades de evidencia, pero no es una memoria persistente, barata ni fiable. Cada consulta larga vuelve a pagar latencia y tokens; además, cuanto más contexto irrelevante se añade, más difícil resulta mantener precisión y trazabilidad.

Problema	Diseño incorrecto	Diseño recomendado
Muchos manuales	Enviar 300 PDFs al prompt.	Indexar y recuperar 5–20 fragmentos relevantes.
Relaciones de componentes	Esperar que el LLM “recuerde” que S se relaciona con barrera y motor.	Persistir esas relaciones y expandirlas antes de buscar.
Casos reales	Pegar todo el historial de tickets.	Memoria episódica estructurada + búsqueda por similitud/entidades.
Cambios de versión	Mezclar procedimientos antiguos y nuevos.	effective_from/effective_to, modelo, firmware y versionado documental.
Conflictos	Dejar que el LLM elija sin saber procedencia.	Detectar conflicto y mostrar fuentes/fechas; priorizar validación.

9. Modelo de conocimiento propuesto para APObot

Para que una pregunta como “me falla el eje S” recorra todo lo relacionado, hay que representar el dominio. Una primera ontología práctica puede ser pequeña y crecer con el uso.

Entidad	Relaciones relevantes
Robot / modelo	TIENE_VERSION, TIENE_EJE, INSTALADO_EN
Eje	TIENE_PARAMETRO, USA_MOTOR, USA_CORREA, INTERACTUA_CON, PRESENTA_SINTOMA
Componente	PARTE_DE, SUSTITUIDO_POR, AFECTA_A
Parámetro	PERTENECE_A, VALIDO_PARA_VERSION, OBTENIDO_DE
Síntoma	PUEDE_INDICAR, OBSERVADO_EN
Avería / diagnóstico	AFECTA_A, SE_RESUELVE_CON, CONFIRMADO_POR
Procedimiento	APLICA_A, REQUIERE, CONTRAINDICADO_EN, FUENTE
Caso real	OBSERVA, DIAGNOSTICA, EJECUTA, RESULTADO, VALIDADO_POR
Documento / página	DESCRIBE, VERSIONA, SUPERA_A

No es necesario introducir Neo4j desde el primer día. Las relaciones pueden vivir inicialmente en PostgreSQL. Neo4j empieza a justificar su coste cuando se necesitan recorridos, explicaciones de dependencias, análisis multi-hop o visualización del conocimiento de forma frecuente.

10. Pipeline de recuperación que recomiendo

Fase	Función	Modelo/tecnología posible
1. Query understanding	Extraer eje, pieza, síntoma, versión, acción y restricciones.	LLM barato o reglas + diccionario técnico.
2. Query expansion	Añadir alias y relaciones del dominio.	SQL/Neo4j + vocabulario/ontología.
3. Lexical retrieval	Capturar nombres exactos, parámetros, códigos y valores.	PostgreSQL FTS/BM25 u OpenSearch.
4. Semantic retrieval	Encontrar reformulaciones y explicaciones equivalentes.	Qwen3-Embedding-4B + pgvector.

Fase	Función	Modelo/tecnología posible
5. Graph retrieval	Traer vecinos relevantes del eje/componente.	SQL relacional o Neo4j 1–2 saltos.
6. Reranking	Elegir evidencia realmente útil.	Qwen3-Reranker-4B / BGE v2-m3.
7. Context builder	Deduplicar, ordenar por fuente/versión y aplicar presupuesto.	Código propio determinista.
8. Generation	Explicar, comparar y proponer pasos.	Router local/API.
9. Verification	Comprobar citas, valores críticos y contradicciones.	Reglas + segundo pase LLM opcional.

11. Router de modelos: la pieza que más ahorra

No todas las preguntas necesitan el mismo modelo. Un router puede decidir el nivel antes o después de la recuperación.

Nivel	Ejemplos	Modelo sugerido
Nivel 0 - determinista	“¿Cuál es KG+ de S?”, lookup de parámetro, listado de piezas.	Sin LLM o respuesta templada desde SQL.
Nivel 1 - barato	Preguntas RAG normales con evidencia clara.	Benchmark: Gemini 2.5 Flash-Lite / Mistral Small 4 / GPT-5.6 Luna / DeepSeek V4.1 Flash.
Nivel 2 - complejo	Síntomas ambiguos, varias averías, versiones y procedimientos multietapa.	Claude Sonnet 5 / GPT-5.6 Terra / Gemini 2.5 Pro / Mistral Medium 3.5.
Nivel 3 - crítico	Conflictos de fuentes, análisis profundo, planificación compleja o caso sin precedentes.	GPT-5.6 Sol / Claude Opus 5 / GPT-6 Astra, con revisión humana.

Este patrón suele producir una mejora de coste mucho mayor que intentar optimizar céntimos en embeddings. Además, mejora la seguridad: los valores simples pueden salir de una fuente estructurada sin dejar que un modelo “recalcule” algo que ya sabemos.

12. Privacidad, seguridad y operación

- Separar el corpus técnico de la generación: los documentos completos pueden permanecer locales y enviar a la API solo los fragmentos necesarios.
- Eliminar secretos, datos personales y credenciales antes de indexar o enviar contexto a terceros.
- Registrar proveedor, modelo, versión, prompt, fuentes recuperadas, latencia y coste por respuesta.
- Aplicar control de acceso por farmacia/cliente/proyecto si la base mezcla datos de instalaciones distintas.
- No guardar automáticamente como “hecho” una afirmación generada por el LLM. La memoria duradera debe proceder de documentos o casos validados.
- Para procedimientos de mantenimiento, mostrar siempre fuente, versión y advertencia de verificación en máquina antes de producción.

13. Roadmap técnico recomendado

Fase A — convertir el MVP en benchmark

- 8. Crear un dataset de 100–300 preguntas reales: literales, reformuladas, multi-hop, ambiguas, fuera de alcance y con conflictos de versión.
- 9. Medir retrieval recall@k, MRR/nDCG, exactitud factual, precisión de fuente, rechazo correcto, latencia y coste.
- 10. Mantener el MVP mínimo como baseline determinista.

Fase B — mejorar recuperación antes del modelo

- 11. Añadir entidades y metadatos técnicos al esquema.
- 12. Introducir embeddings Qwen3 + pgvector y un reranker local.
- 13. Crear búsqueda híbrida y filtros por modelo/versión/fecha.
- 14. Comparar con y sin grafo en preguntas multi-hop.

Fase C — router multi-modelo

- 15. Implementar una interfaz de proveedor independiente.
- 16. A/B test de Nivel 1: Gemini 2.5 Flash-Lite, Mistral Small 4, GPT-5.6 Luna y DeepSeek V4.1 Flash; Nivel 2: Claude Sonnet 5, GPT-5.6 Terra y Gemini 2.5 Pro.
- 17. Definir reglas de escalado por confianza de retrieval, conflicto y complejidad.
- 18. Registrar coste por respuesta y calidad por categoría.

Fase D — memoria de casos reales

- 19. Activar la tabla de casos como memoria episódica versionada y validada.
- 20. Extraer automáticamente entidades y relaciones, pero exigir confirmación para hechos críticos.
- 21. Usar casos resueltos como evidencia adicional, diferenciándolos claramente del manual oficial.

14. Cómo decidir objetivamente qué modelo gana

No escogería el modelo final por benchmarks generales. APObot debe tener su propio benchmark, porque una diferencia pequeña en comprensión de terminología técnica vale más que una gran diferencia en matemáticas o coding.

Métrica	Qué mide	Peso sugerido
Exactitud factual	Valores, parámetros, modelos y pasos correctos.	30%
Precisión de fuente	La cita realmente contiene la evidencia.	20%
Cobertura relacional	Encuentra información relacionada aunque no aparezca literal en la pregunta.	15%
Rechazo correcto	No inventa cuando falta evidencia.	10%
Claridad técnica	Explica en orden y con condiciones.	10%
Latencia p95	Experiencia operativa.	5%
Coste por caso resuelto	Coste real, incluyendo escalados.	10%

La métrica decisiva debe ser “coste por respuesta correcta y trazable”, no “precio por millón de tokens”. Un modelo barato que obliga a reintentar o escala constantemente puede salir más caro que uno intermedio.

15. Propuesta concreta para el próximo APObot

Bloque	Elección inicial	Motivo
API / backend	ASP.NET Core .NET 10	Continuidad con el stack actual.
Base operativa	PostgreSQL	Casos, hechos, versiones y permisos.
Vectorial	pgvector	Evita añadir otro servicio mientras el tamaño lo permita.
Full-text	PostgreSQL FTS inicialmente	Excelente para códigos/parámetros exactos; migrar a OpenSearch si corpus/tráfico lo exige.
Embeddings	Qwen3-Embedding-4B local	Buena comprensión multilingüe y coste marginal local.
Reranker	Qwen3-Reranker-4B o BGE v2-m3	Reduce contexto enviado y mejora relevancia.
Grafo	Relaciones en PostgreSQL -> Neo4j si aporta métrica	Evita complejidad prematura.
LLM Nivel 1	A/B: Gemini 2.5 Flash-Lite / Mistral Small 4 / GPT-5.6 Luna	Costes mínimos; seleccionar por benchmark propio. DeepSeek V4.1 Flash como benchmark adicional.
LLM Nivel 2	A/B: Claude Sonnet 5 / GPT-5.6 Terra / Gemini 2.5 Pro	Equilibrio de coste, contexto y razonamiento.
LLM Nivel 3	GPT-5.6 Sol / Claude Opus 5 / GPT-6 Astra	Solo escalado selectivo y trazado.
Local fallback	Qwen3 14B o 30B-A3B	Modo offline, privacidad y continuidad.

Resultado esperado: APObot puede crecer desde un manual a una plataforma con miles de documentos y casos sin depender de que un LLM “se acuerde” de todo. El modelo se convierte en una capa sustituible; el activo real es la memoria estructurada, la calidad de recuperación y la trazabilidad.

16. API keys y facturación: qué se paga realmente

Una API key no es una suscripción ni un modelo: es la credencial con la que APObot autentica sus llamadas. El coste real aparece por el uso asociado a esa clave: tokens de entrada/salida, embeddings, almacenamiento, herramientas, búsquedas, caché, procesamiento regional o capacidad reservada. En OpenAI, la facturación de ChatGPT y la plataforma API es independiente: tener ChatGPT Plus/Pro no incluye consumo de API [S14].

Para producción conviene crear claves separadas por entorno (desarrollo, staging, producción), nunca guardarlas en GitHub, usar variables de entorno o un gestor de secretos, fijar presupuestos/alertas y rotarlas. Así se puede atribuir el coste por servicio, detectar fugas y revocar una clave sin interrumpir todo APObot.

Proveedor	Qué se paga	Servicios relevantes	Ahorros / recargos	Nota para APObot
OpenAI	Sin cuota separada por la clave; se factura el uso de API	GPT-5.6 / GPT-6 + embeddings + tools	Batch ~50%; input cacheado muy rebajado; regional/fast pueden añadir recargos	ChatGPT y API se facturan por separado [S14][S15].
Anthropic	Credencial de Claude Platform; coste por consumo	Haiku 4.5 / Sonnet 5 / Opus 5	Prompt caching hasta ~90%; batch ~50% [S3]	Adecuado para routing multi-modelo; controlar cache writes/hits.
Google Gemini	Clave del Gemini Developer API; free tier y paid tier	Flash-Lite / Flash / Pro + Embedding 2	Batch ~50%; context caching con tarifa propia [S4]	Paid tier: datos no usados para mejorar productos según la página de precios.
Mistral	Clave de API; facturación por uso	Small 4 / Large 3 / Medium 3.5 / Embed	Batch 50%; cached input hasta -90%; inferencia regional +10% [S5]	Interesante para coste bajo y opción europea.
DeepSeek	Clave de API y saldo/cobro por tokens	V4.1 Flash	Off-peak = 50% del peak; cache hit muy barato [S13]	Usarlo como benchmark económico; revisar requisitos de datos antes de producción.

17. Costes operativos que faltaban: embeddings, caché, batch y RAG gestionado

En un RAG grande, la generación es solo una parte de la factura. Embeddings e indexación suelen ser baratos; el mayor coste recurrente aparece en generación, contexto innecesario, herramientas y servicios gestionados. Por eso reducir el contexto mediante recuperación y reranking tiene más impacto económico que abaratar céntimos en la indexación.

17.1 Coste de embeddings e indexación

Embedding	Precio de referencia	Indexar 100M tokens	Lectura económica
Qwen3-Embedding local	Sin tarifa API	Hardware/electricidad	Coste marginal muy bajo; opción recomendada si la infraestructura ya existe.
OpenAI text-embedding-3-small	\$0,02 / 1M tokens [S16]	\$2	Extremadamente barato para texto; baseline API.

Embedding	Precio de referencia	Indexar 100M tokens	Lectura económica
OpenAI text-embedding-3-large	\$0,13 / 1M tokens [S16]	\$13	Más potente/multilingüe; dimensiones configurables.
Mistral Embed	\$0,10 / 1M tokens [S17]	\$10	Alternativa económica; contexto 8K en la ficha consultada.
Gemini Embedding 2	\$0,20 / 1M tokens estándar [S18]	\$20	Multimodal: texto, imágenes, audio, vídeo y PDF; batch ~50%.

Conclusión: incluso un corpus inicial de 100 millones de tokens puede costar solo unos pocos o decenas de dólares en embeddings API. No merece la pena sacrificar calidad de recuperación por ahorrar aquí sin medirlo; el verdadero gasto es volver a enviar demasiado contexto al LLM.

17.2 Caché y batch: descuentos que cambian el coste real

Proveedor	Caché	Batch / horario	Implicación
OpenAI	GPT-5.6 Luna: \$0,20 input vs \$0,02 cached; Terra \$2 vs \$0,20; Sol \$4 vs \$0,40 [S1]	Batch: ~50% [S15]	Muy útil para instrucciones, esquema y contexto repetido.
Anthropic	Prompt caching: hasta ~90% de ahorro en hits [S3]	Batch: ~50% [S3]	Importante en manuales/PDF repetidos y conversaciones largas.
Google	Context caching con precio específico; Gemini 3.8 Flash \$0,075/M durante 2026 frente a \$0,75 input [S4]	Batch ~50% [S4]	La caché tiene también coste de almacenamiento por hora.
Mistral	Cached input hasta -90% [S5]	Batch 50% [S5]	Muy interesante para prompts RAG con prefijos estables.
DeepSeek V4.1 Flash	\$0,003/M cache-hit off-peak vs \$0,15/M cache-miss; peak \$0,006 vs \$0,30 [S13]	Off-peak 50% del peak	Puede reducir mucho cargas programables o agentes con contexto repetido.

17.3 RAG gestionado frente a pgvector propio

Opción	Coste publicado / tipo de coste	Ventaja	Papel sugerido
PostgreSQL + pgvector propio	Software open source; coste = servidor/DB + operación	Control total de chunks, filtros, versiones, grafo y reranker.	Recomendado para el núcleo de memoria APObot.
OpenAI File Search	\$0,10/GB/día de vector store (primer GB gratis) + \$2,50/1.000 tool calls [S19]	Reduce trabajo de infraestructura, pero añade coste recurrente y menos control.	Útil para prototipos o subsistemas; no lo usaría como única memoria.
Google File Search	Almacenamiento y embeddings de consulta sin coste; indexación se cobra por embeddings y tokens recuperados a tarifa normal [S20]	Servicio gestionado con menos operación.	Interesante como comparación de TCO si se usa Gemini.

Ejemplo OpenAI File Search: 10 GB de vector store sostenidos durante 30 días implican aproximadamente \$27/mes de almacenamiento tras descontar el primer GB gratuito; 100 GB, aproximadamente \$297/mes, antes de tool calls y tokens. En pgvector propio el software no cobra por GB, pero sí existe coste de infraestructura y mantenimiento.

17.4 Coste de un router multi-modelo

Estrategia	1.000 consultas	50.000 consultas	500.000 consultas
Todo GPT-5.6 Luna	\$2.04	\$102	\$1020
Todo Claude Sonnet 5	\$19.00	\$950	\$9500
Todo GPT-5.6 Sol	\$38.00	\$1900	\$19000
Router 90% Luna / 8% Sonnet 5 / 2% Sol	\$4.12	\$206	\$2058
Router 90% Mistral Small 4 / 8% Sonnet 5 / 2% Sol	\$3.47	\$173	\$1734

El router no pretende ser más barato que usar siempre el modelo barato; compra calidad de forma selectiva. En este ejemplo, el router Luna -> Sonnet -> Sol cuesta ~\$4,12 por 1.000 consultas, pero sigue siendo unas 4,6 veces más barato que enviar todo a Sonnet 5 y unas 9,2 veces más barato que enviar todo a Sol. La proporción real debe salir del benchmark APObot, no fijarse a priori.

17.5 Recomendación coste-beneficio revisada

- 1) Mantener memoria, relaciones, FTS, pgvector y reranking bajo nuestro control. Así el LLM se convierte en una capa sustituible.
- 2) Nivel 1: medir Gemini 2.5 Flash-Lite, Mistral Small 4, GPT-5.6 Luna y DeepSeek V4.1 Flash. El ganador debe ser el menor coste por respuesta correcta y trazable, no el menor \$/M tokens.
- 3) Nivel 2: medir Claude Sonnet 5, GPT-5.6 Terra y Gemini 2.5 Pro sobre preguntas ambiguas, multi-hop y con conflictos de versión.
- 4) Nivel 3: reservar GPT-5.6 Sol, Claude Opus 5 y GPT-6 Astra para una fracción pequeña de consultas, idealmente con revisión humana cuando la intervención sea crítica.
- 5) Explotar caché y batch donde encajen. Para ingesta, reindexación, clasificación offline y generación de resúmenes, batch puede reducir aproximadamente a la mitad la factura en varios proveedores.
- 6) No considerar “gratuito” un modelo local: no cobra tokens, pero sí GPU/CPU, RAM, electricidad, mantenimiento, latencia y capacidad. A volúmenes pequeños/medios, una API barata puede costar menos que comprar hardware solo para inferencia.

18. Fuentes y precios consultados

Precios y características cambian con rapidez. Los importes de este documento son una fotografía del 17 de septiembre de 2026. Antes de contratar producción deben verificarse de nuevo, especialmente promociones, contexto largo, batch, caché, residencia regional, herramientas y tarifas de almacenamiento.

[S1] OpenAI API - modelos y precios GPT-5.6: <https://openai.com/es-ES/api/>

[S2] OpenAI GPT-5.6 Luna - ficha de modelo y cached input:
<https://developers.openai.com/api/docs/models/gpt-5.6-luna>

[S3] Anthropic - Sonnet 5, Opus 5 y Haiku 4.5: <https://www.anthropic.com/news/claude-sonnet-5> ;
<https://www.anthropic.com/claude/opus> ; <https://www.anthropic.com/claude/haiku>

[S4] Google Gemini Developer API - pricing: <https://ai.google.dev/gemini-api/docs/pricing>

[S5] Mistral API - pricing: <https://docs.mistral.ai/inference/pricing>

[S6] Qwen — Qwen3 official blog, tamaños y ventanas de contexto. [Fuente](#)

[S7] Qwen — Qwen3 Embedding y Reranker. [Fuente](#)

[S8] Google DeepMind — Gemma 3 model card. [Fuente](#)

[S9] BAAI — bge-reranker-v2-m3. [Fuente](#)

[S10] OpenAI embeddings: <https://developers.openai.com/api/docs/models/text-embedding-3-small> ;
<https://developers.openai.com/api/docs/models/text-embedding-3-large>

[S11] Google embeddings: <https://ai.google.dev/gemini-api/docs/embeddings>

[Proyecto] Repositorio APObot Knowledge MVPs usado como punto de partida. [Fuente](#)

[S12] OpenAI GPT-6 Astra - pricing/model card: <https://developers.openai.com/api/docs/models/gpt-6-astra>

[S13] DeepSeek API - Models & Pricing (V4.1 Flash): https://api-docs.deepseek.com/quick_start/pricing/

[S14] OpenAI - ChatGPT y API tienen facturación separada: <https://help.openai.com/en/articles/9039756>

[S15] OpenAI Batch API - descuento del 50%: <https://help.openai.com/en/articles/9197833>

[S16] OpenAI embeddings - \$0,02/M small y \$0,13/M large:
<https://developers.openai.com/api/docs/models/text-embedding-3-small>

[S17] Mistral Embed - \$0,10/M tokens: <https://docs.mistral.ai/models/mistral-embed-23-12>

[S18] Gemini Embedding 2 - pricing multimodal: <https://ai.google.dev/gemini-api/docs/pricing>

[S19] OpenAI File Search - almacenamiento y tool calls: <https://openai.com/index/new-tools-and-features-in-the-responses-api/>

[S20] Gemini File Search - costes de indexación/almacenamiento: <https://ai.google.dev/gemini-api/docs/file-search>

Anexo A. Fórmula de coste usada

Coste por consulta = (tokens de entrada / 1.000.000 × precio de entrada) + (tokens de salida / 1.000.000 × precio de salida).

Escenario comparativo: 6.000 tokens de entrada + 700 tokens de salida. Ejemplo GPT-5.6 Luna: $0,006 \times 0,20 \text{ USD} + 0,0007 \times 1,20 \text{ USD} = 0,00204 \text{ USD}$ por consulta; aproximadamente 2,04 USD por 1.000 consultas.

La tabla principal de coste por consulta no aplica descuentos de caché, batch, compromisos empresariales ni recargos regionales. Las secciones 16-17 muestran cómo esos factores pueden cambiar el TCO real. Los impuestos (IVA u otros) tampoco están incluidos.

Anexo B. Fórmulas rápidas para presupuestar APObot

Generación: $\text{coste} = (\text{tokens_entrada} / 1.000.000 \times \text{tarifa_entrada}) + (\text{tokens_salida} / 1.000.000 \times \text{tarifa_salida})$.

Embeddings: $\text{coste} = \text{tokens_indexados} / 1.000.000 \times \text{tarifa_embedding}$. La reindexación completa vuelve a pagar ese volumen salvo que se use hash/incrementalidad.

Router: $\text{coste medio} = \Sigma (\text{porcentaje_de_tráfico_nivel} \times \text{coste_por_consulta_nivel})$. Añadir por separado herramientas, almacenamiento, búsquedas y reintentos.

TCO local anual = $\text{amortización_hardware} + \text{electricidad} + \text{operación/monitorización} + \text{copias} + \text{coste de oportunidad}$. “0 \$/token” no equivale a “0 \$ de coste”.